Code Review - Group 1: Smart Campus Marketplace

1. Metadata

- Group: 1
- Project: Smart Campus Marketplace
- Members:
  - 2301140001 - Nguyễn Quốc An
  - 2301140016 - Nguyễn Trí Dũng
  - 2301140057 - Hoàng Phước Long
- Review date: 2026-05-19
- Review scope:
  - `/home/tamnt/Documents/4th year/ss2/code/group1/campus`
  - Commit `5cfc89c0` ("final1", 2026-05-18)
  - Branch `main`

2. Executive summary

Key risks

1. **Broken conversation authorization → IDOR (Critical).** `controllers/chat/index.js:77` and `:110` use `conv.participants.includes(userId)` where both sides are Mongoose `ObjectId` objects — `Array.prototype.includes` uses reference equality, so the guard never holds. Any logged-in user can read/post into any conversation by id. See SEC-1.
2. **Multi-document money flows have no transactions (High).** Order creation (`controllers/orders/index.js:106-229`), payment confirmation (`controllers/checkout/payment.js:78-120`) and wallet settlement (`controllers/orders/index.js:481-518`) write to 4–6 collections with manual best-effort rollback only; a crash mid-flow leaves wallets/orders/products inconsistent. See DB-3.
3. **Double-credit race on the seller wallet (Medium-High).** The webhook and the SePay-poll path both run a read-modify-write `wallet.pendingBalance += payment.amount; await wallet.save()` with no atomic guard. See DB-4.
4. **No automated tests at all**, yet CI runs `npx playwright test` with no suite/config — pipeline is misleading and there is zero regression safety on payment code. See TEST-1.

5. **Stored XSS via raw `<%- JSON.stringify(userData) %>` inside `<script>`** (`views/product.ejs:391-393`, `views/order-tracking.ejs:1106`, `views/sell.ejs:1272`). See FE-1.
6. **Frontend is the weakest area:** only 1 of 23 views uses the `head.ejs` partial, 800–1400-line single-file templates with ~500-line inline scripts, `escapeHtml`/currency helpers copy-pasted 8–14×, a broken/dead `public/js/auth.js`, and `localStorage`-based role/mode decisions. See section 6.11.

3. Rubric overview

Category	Score	Notes
Folder organization	6/10	Clean domain-grouped controllers/models/routes; loses points for committed scratch/tmp files (FO-1), a fat-controller pattern with no service layer (FO-2), and a disorganized frontend (dead `public/js/auth.js`, `head.ejs` used by 1/23 views, 19 ad-hoc CSS files).
Architecture	4/10	Conventional route→controller→model with no service/repository layer; compensating logic duplicated across 3 files (ARCH-1). Frontend has effectively no architecture: no layout system, 800–1400-line templates with ~500-line inline scripts, broken dead client modules (ARCH-3/ARCH-4).
API design	6/10	Consistent `{success,data}` envelope and pagination; but `PATCH /products/:id` overloaded with side effects (API-1), inconsistent 4xx/5xx (API-2), and `googleId` leaked because `.lean()` bypasses `toJSON` (API-3).
Database & queries	5/10	Reasonable compound indexes and `.lean()`/projection discipline; undercut by N+1 in chat inbox (DB-1) and rating recompute (DB-2), no transactions on multi-collection writes (DB-3), wallet race (DB-4), and undisciplined index declarations — duplicate unique indexes, low-cardinality singletons, prod `autoIndex` on (DB-IDX-1).
Performance	5/10	Server read paths paginated, but the home page over-fetches 100 products and filters client-side (PERF-3), no build/`defer`, floating-version CDNs; `syncAllRatings`/chat loops scale poorly (PERF-1); AI/SePay uncached (PERF-2).
Constants/enums/config	8/10	Strong centralization in `config/appConstants.js`; payment statuses (`PENDING/PAID/EXPIRED`) and `'admin'`/`'qr'` literals bypass the enums (CONST-1).

Category	Score	Notes
Validation/error handling	5/10	Ad-hoc manual validation, no schema validator (VAL-1); raw `err.message` returned to clients across nearly every catch (VAL-2).
Security	4/10	OAuth+JWT+httpOnly cookie and timing-safe webhook are good; undercut by broken chat authz / IDOR (SEC-1), no CSRF on cookie-auth routes (SEC-2), wildcard Socket.IO CORS (SEC-3), unscoped `getPaymentStatus` (SEC-4), plus stored XSS (FE-1) and client-side role checks (FE-2).
Documentation/DX	8/10	Excellent `README.md`/`ENV_SETUP.md` and `dbdiagram.dbml`; no committed `.env.example` though `README.md:58` says `cp .env.example .env` (DOC-1).

4. Detailed s

Part I — Frontend

4.1 Architecture (frontend)

No frontend architecture — 23 monolithic templates, layout partial used by 1 view

- Severity: High
- Evidence:
  - `views/partials/head.ejs` exists but is included by **only** `views/search-results.ejs` (verified: `grep -rl "partials/head" views/` returns 1 file); every other view hand-duplicates `<!DOCTYPE>`, `<head>`, font/CDN `<link>`s.
  - Largest views are unmanageable single files: `views/dashboard-admin.ejs` ~1401 lines, `views/order-tracking.ejs` ~1365, `views/sell.ejs` ~1285, `views/index.ejs` 778 lines with a ~495-line inline `<script>` (lines ~280-775).

Current code — `views/index.ejs:1-11` (the boilerplate copy-pasted into ~22 views instead of `<%- include('partials/head') %>`):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Campus Marketplace — Buy &amp; Sell Student Goods</title>
  <link rel="stylesheet" href="/css/shared.css">
  <link rel="stylesheet" href="/css/main.css">
  <link rel="stylesheet" href="/css/index.css">
```

```
  <link rel="stylesheet" href="/css/rating.css">
</head>
```

- Issue: The one DRY layout artifact (`head.ejs`) is essentially dead — 22 of 23 views never include it; every template is a standalone HTML document with embedded CSS and a large inline script.
- Impact: Changing the font, a meta tag, a CSS path, or adding analytics requires editing ~22 files; templates are too large to review/test; bugs are fixed per page (this is the root cause of the FE-3 helper duplication).

Suggested fix — adopt a layout and a parametrized head partial:

```
<!-- views/partials/head.ejs -->
<!DOCTYPE html><html lang="en"><head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title><%= typeof pageTitle !== 'undefined' ? pageTitle : 'Campus Marketplace'
%></title>
  <link rel="stylesheet" href="/css/shared.css">
  <link rel="stylesheet" href="/css/main.css">
  <% (typeof pageStyles==='undefined'?[]:pageStyles).forEach(s => { %>
    <link rel="stylesheet" href="/css/<%= s %>.css"><% }) %>
</head>
```

```
<%- include('partials/head', { pageTitle: 'Campus Marketplace', pageStyles:
['index','rating'] }) %>
```

Every view includes the partial; one edit changes the head everywhere, and per-page inline scripts move into `public/js/pages/<view>.js`.

ARCH-4: Dead / broken client module shipped (`public/js/auth.js`)

- Severity: Medium
- Evidence: `public/js/auth.js:1` vs `public/js/config.js:2-12`

Current code — `public/js/auth.js:1-14`:

```
import { API_BASE, API_URL, STORAGE_KEYS, ROUTES } from './config.js';
import { api } from './api.js';

export const auth = {
  getToken()  { return localStorage.getItem(STORAGE_KEYS.TOKEN); },
  getUser() {
    const raw = localStorage.getItem(STORAGE_KEYS.USER);
    return raw ? JSON.parse(raw) : null;
  },
```

```
  isAuthenticated() { return !!this.getToken() && !!this.getUser(); },
  isAdmin() { return this.getUser()?.role === 'admin'; },
```

Current code — `public/js/config.js:1-12` (only `API_BASE` and `ROUTES` are exported — `API_URL` and `STORAGE_KEYS` do not exist):

```
export const API_BASE = '/api';
export const ROUTES = {
  HOME: '/', LOGIN: '/login', LOGOUT: '/logout',
  SELL: '/sell', MY_PRODUCTS: '/my-products', DASHBOARD: '/dashboard',
};
```

- Issue: `auth.js` imports `API_URL` and `STORAGE_KEYS` that `config.js` never exports, so they are `undefined`; `getToken()` evaluates `localStorage.getItem(undefined.TOKEN)` → `TypeError`. The module also assumes localStorage-token auth, but the app moved to httpOnly cookies (`controllers/auth/index.js:10-17`). No view imports `auth.js`.
- Impact: Dead, misleading code that crashes on first use and contradicts the real cookie-auth model; if someone wires it up it throws immediately.

Suggested fix — delete the broken module and centralize the real session check:

```
// public/js/session.js
export async function getCurrentUser() {
  const r = await fetch('/api/auth/me', { credentials: 'include' });
  if (!r.ok) return null;
  return (await r.json()).data;
}
export const isAdmin = (u) => u?.role === ROLE_LIST.ADMIN;
```

4.2 Performance (frontend)

PERF-3: Home page over-fetches 100 products and filters client-side; no build / `defer`

- Severity: Medium
- Evidence: `views/index.ejs:332`, `views/index.ejs:276-279`

Current code — `views/index.ejs:332-334` (pulls up to 100 full product docs, then all filter/sort/paginate is in-memory):

```
async function fetchProducts() {
  try {
    const res = await fetch('/api/products?limit=100', { credentials:
'include' });
```

```
    const json = await res.json();
    const products = json.data || [];
```

Current code — `views/index.ejs:276-279` (render-blocking floating-version CDN + no `defer` on any script):

```
<!-- Lucide Icons CDN -->
<script src="https://unpkg.com/lucide@latest/dist/umd/lucide.min.js"></script>
<script src="/js/categories.js"></script>
<script src="/js/rating.js"></script>
```

- Issue: The landing page bypasses the server pagination that `controllers/product/index.js:43-54` already implements and downloads 100 docs; scripts load synchronously from an unpinned `@latest` CDN with no `defer`.
- Impact: Large transfer + layout shift on the most-visited page; first paint blocked on `unpkg`; a breaking `@latest` release or unreachable CDN breaks the home page.

Suggested fix — use server pagination + non-blocking scripts:

```
const params = new URLSearchParams({ page: state.page, limit: 12, sort:
state.sort, ...state.filters });
const res = await fetch(`/api/products?${params}`, { credentials: 'include' });

<script defer src="/vendor/lucide-0.460.0.min.js"></script>
<script defer src="/js/categories.js"></script>
```

Self-host/version-pin Lucide, `defer` all scripts, and let the server page the grid (12 at a time) instead of shipping 100 docs.

4.3 Frontend

FE-1: Stored XSS via raw `<%- JSON.stringify(userData) %>` inside `<script>`

- Severity: High
- Evidence: `views/product.ejs:391-393`, `views/order-tracking.ejs:1106`, `views/sell.ejs:1272`

Current code — `views/product.ejs:389-394`:

```
const PRODUCT_META = {
  id: PRODUCT_ID,
  title: <%- JSON.stringify(product.title) %>,
  price: <%- JSON.stringify(fmtPrice(product.price) + ' VND') %>,
  image: <%- JSON.stringify((product.images && product.images[0]) || '') %>,
```

```
  url: '/products/' + PRODUCT_ID
};
```

Current code — `views/order-tracking.ejs:1106`:

```
const order = <%- JSON.stringify(order) %>;
```

- Issue: `<%- %>` is unescaped output and `JSON.stringify` does **not** escape `<`, `>` or `/`. A seller-controlled `product.title` containing `</script><img src=x onerror=alert(document.cookie)>` terminates the inline `<script>` element at the HTML-parser level; `JSON.stringify` provides zero protection here. `order` (line 1106) embeds buyer addresses/messages the same way.
- Impact: A seller can store a payload in a product title/description that executes JS in every authenticated buyer who opens `/products/:id` or the order-tracking page — session riding, message theft, order actions on the victim's behalf.

Suggested fix — escape for the HTML/script context (or use a JSON `<script>` block):

```
// views/product.ejs
const PRODUCT_META = JSON.parse(<%-
  JSON.stringify(JSON.stringify({
    id: String(product._id), title: product.title,
    price: fmtPrice(product.price) + ' VND',
    image: (product.images && product.images[0]) || '',
  })).replace(/</g, '\\u003c').replace(/>/g, '\\u003e').replace(/&/g, '\\u0026')
%>);
```

Or render `<script id="product-meta" type="application/json"><%- ... %></script>` and `JSON.parse(el.textContent)`. Either way the data can never break out of script context.

FE-2: Authorization/role decisions made in client-side JavaScript

- Severity: Medium
- Evidence: `public/js/search.js:53-66`, `views/messages.ejs:241`, `public/js/auth.js:14`

Current code — `public/js/search.js:53-66`:

```
function getItemsByRole() {
    let currentRole = 'buyer';
    try {
        if (window.location.pathname.startsWith('/admin')) {
            return ALL_ITEMS.filter(item => item.role === 'admin' || !item.role);
        }
        const mode = localStorage.getItem('campus_mode') || 'buyer';
```

```
        currentRole = mode;
    } catch (e) {}
    return ALL_ITEMS.filter(item => !item.role || item.role === currentRole);
}
```

Current code — `views/messages.ejs:241-242` (which conversations a user "sees" is a client-side filter keyed on a user-writable localStorage value):

```
const uiMode = (localStorage.getItem('campus_mode') || 'buyer');
document.getElementById('mode-label').textContent = uiMode === 'seller' ? 'Seller
view' : 'Buyer view'; -> define enum type not use hard code literal string
```

- Issue: Role/mode is read from `localStorage` (fully user-controllable via devtools) and used to choose navigation items and which conversations to display. Access decisions belong on the server.
- Impact: A user can set `localStorage.campus_mode='seller'` to surface seller navigation; the messages "mode" split is cosmetic and depends on the API already scoping data — which it does not for chat (see SEC-1). Encourages a false sense of access control.

Suggested fix — treat client mode as presentation only; scope on the server:

```
// server returns only what the user may see
// controllers/chat/index.js getConversations already filters by participants;
// expose role from the server, never derive it from localStorage:
const user = await fetch('/api/auth/me', { credentials:'include' }).then(r =>
r.json());
const isAdmin = user.data?.role === ROLE_LIST.ADMIN';   // server-asserted
```

=> all the authorizations related to security must be in the backend not in frontend

FE-3: `escapeHtml`/currency helpers copy-pasted across 8–14 files (no shared util)

- Severity: Medium
- Evidence: `escapeHtml`/`escHtml` redefined in `views/index.ejs:379`, `views/my-products.ejs:148`, `views/favorites.ejs:162`, `views/admin-disputes.ejs:149`, `views/notifications.ejs:78`, `public/js/notifications.js:100`, `public/js/chat-dropdown.js:204`, `public/js/rating.js:348` (8 copies); `Intl.NumberFormat(... 'VND')` re-implemented in 14 views.

Current code — `views/index.ejs:379-381`:

```
function escapeHtml(s) {
  return String(s||'').replace(/&/g,'&amp;').replace(/</g,'&lt;').replace(/
>/g,'&gt;').replace(/"/g,'&quot;');
}
```

Current code — `views/my-products.ejs:148` (different coverage — this one *does* escape `'`):

```
function escapeHtml(s) { return (s || '').replace(/[&<>"']/g, c => ({ '&':
'&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' })[c]); }
```

Current code — `public/js/rating.js:348-352` (a *third*, semantically different implementation using a DOM node):

```
function escapeHtml(text) {
  const div = document.createElement('div');
  div.textContent = text;
  return div.innerHTML;
}
```

- Issue: The same display-critical helper exists in 8 places with divergent behavior — `index.ejs` does **not** escape `'`, `my-products.ejs` does, `rating.js` uses a different algorithm entirely. Currency formatting is duplicated in 14 templates.
- Impact: Inconsistent escaping/formatting across pages; a security fix (e.g. the FE-1 `</>` hardening, or escaping `'`) must be applied 8+ times and will be missed somewhere.

Suggested fix — one shared module imported everywhere:

```
// public/js/utils.js
export const escapeHtml = (s) =>
  String(s ?? '').replace(/[&<>"'`]/g, c =>
    ({ '&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;',"'":'&#39;','`':'&#96;' }
[c]));
export const formatVND = (n) =>
  new Intl.NumberFormat('vi-VN', { style:'currency',
currency:'VND' }).format(Number(n) || 0);

import { escapeHtml, formatVND } from '/js/utils.js';
```

Delete the 8 `escapeHtml`/`escHtml` redefinitions and 14 inline formatters; one fix now propagates everywhere.

FE-4: ~122 inline `onclick=` handlers with EJS values interpolated into attributes

- Severity: Medium
- Evidence: 122 `onclick=` occurrences across `views/*.ejs` (verified count)

Current code — `views/index.ejs:124-129`:

```
<button class="filter-btn active" data-filter="all"
onclick="setFilter(this,'all')">
```

```
  <i data-lucide="layout-grid" width="14" height="14"></i> All
</button>
<% categories.forEach(cat => { %>
  <button class="filter-btn" data-filter="<%= cat.slug %>"
onclick="setFilter(this,'<%= cat.slug %>')">
    <i data-lucide="<%= cat.lucideIcon %>" width="14" height="14"></i>
```

- Issue: Behavior is wired via HTML attributes calling globals, with EJS values interpolated directly into the JS string in the attribute (`onclick="setFilter(this,'<%= cat.slug %>')"`) — data, markup and behavior fully entangled across ~122 sites.
- Impact: Untestable handlers, global-namespace collisions across many page scripts, attribute-injection risk wherever an interpolated value is not strictly slug-safe, and it blocks any future Content-Security-Policy (inline handlers require `unsafe-inline`).

Suggested fix — delegated listeners + `data-*`:

```
<button class="filter-btn" data-action="filter" data-filter="<%= cat.slug
%>"></button>

document.querySelector('.filter-bar').addEventListener('click', (e) => {
  const btn = e.target.closest('[data-action="filter"]');
  if (btn) setFilter(btn, btn.dataset.filter);
});
```

Removes all inline handlers (a prerequisite for a CSP that pairs with SEC-2/3) and makes wiring testable.

FE-5: No distinct loading / empty / error states; failures only `console.error`

- Severity: Medium
- Evidence: `views/index.ejs:355-362`

Current code — `views/index.ejs:355-362`:

```
} catch (err) {
  console.error('Error loading products:', err);
  buildSearchCategoryOptions([]);
  renderSearchCategoryMenu();
  setSearchCategory(searchCatSelect.value || '');
  renderProducts([]);
}
```

- Issue: A failed catalog fetch logs to console and then `renderProducts([])` — making a network error visually identical to "no products for sale". There is no error banner or retry path.
- Impact: On a flaky network the homepage looks like an empty marketplace; the user gets no feedback or way to recover; perceived quality is poor.

Suggested fix — three explicit states with retry:

```
try {
  showState('loading');
  const data = await fetchProducts();
  showState(data.length ? 'ready' : 'empty');
  renderProducts(data);
} catch (err) {
  console.error(err);
  showState('error');                  // banner: "Couldn't load products"
  document.getElementById('retry').onclick = loadCatalog;
}
```

Distinguishing loading/empty/error (ideally in the shared render util from FE-3) gives users feedback and a retry.

FE-6: Per-page `<style>` blocks and 19 hand-written stylesheets alongside Tailwind

- Severity: Low
- Evidence: `public/css/` contains 19 stylesheets (`shared.css`, `main.css`, `index.css`, `product.css`, `checkout.css`, …); `views/index.ejs:8-11` loads several individually; `tailwind.config.js` scans `views/**/*.ejs` but templates use custom classes.

Current code — `views/index.ejs:7-11`:

```
<link rel="stylesheet" href="/css/shared.css">
<link rel="stylesheet" href="/css/main.css">
<link rel="stylesheet" href="/css/index.css">
<link rel="stylesheet" href="/css/rating.css">
```

- Issue: Three competing styling strategies coexist — Tailwind, 19 per-page hand CSS files, and inline `<style>` blocks — with hard-coded hex values (`#1B5EFF`, `#8890B0`) repeated across files instead of design tokens.
- Impact: Unpredictable cascade/specificity, duplicated color constants, larger combined CSS payload, hard to retheme; Tailwind's value is largely unrealized.

Suggested fix — commit to one approach with tokens:

```
/* public/css/tailwind-input.css */
@layer base { :root { --brand: #1B5EFF; --muted: #8890B0; } }
@layer components { .btn-primary { @apply bg-[var(--brand)] text-white rounded-lg
px-4 py-2; } }
```

Drive the head partial (ARCH-3) to load only `tailwind.css` + one optional page sheet; centralize colors as CSS variables / Tailwind theme tokens and delete inline `<style>` blocks.

Part II — Backend

4.4 Folder organization (backend)

FO-2: No service layer — business logic concentrated in controllers

- Severity: Medium
- Evidence: `controllers/orders/index.js:15-274` (`createOrder` is one 260-line function), `controllers/rating/index.js:9-150` (rating math + seller re-aggregation inline)

Current code (`controllers/orders/index.js:170-229`, HTTP handler also doing counters, conversation, auto-message and notification inline):

```
Promise.all([
  User.findByIdAndUpdate(sellerId, { $inc: { totalSales: orderQuantity } }),
  User.findByIdAndUpdate(buyerId, { $inc: { totalOrders: 1 } })
]).catch(err => console.error('[orders] counter update error:', err));

const product = claimedProduct;
let conv = null;
try {
  conv = await findOrCreateConversation(buyerId, sellerId, productId);
  // ...build autoMsg, Message.create, socket emit, conv.save, sendNotification...
} catch (chatErr) {
  console.error('[checkout] Auto-message error:', chatErr);
}
```

- Issue: The controller mixes HTTP parsing, stock claim, payment creation, counter updates, conversation/messaging and notifications in one function; there is no reusable `orderService`.
- Impact: This logic is untestable in isolation (TEST-1 has zero coverage precisely because of this), and the stock-restore code is duplicated into `checkout/payment.js` and `orders/dispute.js` (see ARCH-1).

Suggested fix — extract a service the controller calls:

```
// services/orderService.js
exports.createOrder = async ({ buyerId, dto }) => {
  const product = await claimStock(dto.productId, buyerId, dto.quantity); //
throws DomainError
  const order   = await Order.create(buildOrderDoc(product, buyerId, dto));
  const payment = dto.paymentMode === 'qr' ? await createSePayPayment(order) :
null;
  await Promise.allSettled([bumpCounters(order), createOrderConversation(order,
product)]);
  return { order, payment };
```

```
};

// controllers/orders/index.js
exports.createOrder = async (req, res, next) => {
  try {
    const { order, payment } = await orderService.createOrder({ buyerId:
req.user._id, dto: req.body });
    res.status(201).json({ success: true, orderId: order._id, paymentId:
payment?._id ?? null });
  } catch (err) { next(err); }
};
```

The controller becomes a thin HTTP adapter and the order flow becomes unit-testable without Express.

4.5 Architecture (backend)

ARCH-1: Stock-restore / cancel logic duplicated across 3 modules

- Severity: Medium
- Evidence:
  - `controllers/orders/index.js:428-436` (cancel branch in `updateOrderStatus`)
  - `controllers/checkout/payment.js:25-33` (`cancelOrderAndRestoreStock`)
  - `controllers/orders/dispute.js:184-185` (buyer-favor refund release)

Current code — `controllers/orders/index.js:428-436`:

```
if (status === ORDER_STATUS.CANCELLED && prevStatus !== ORDER_STATUS.CANCELLED) {
  await Product.findByIdAndUpdate(updatedOrder.product, {
    $inc: { quantity: updatedOrder.quantity || 1 },
    $set: { status: PRODUCT_STATUS.ACTIVE, buyer: null, soldAt: null }
  });
  Promise.all([
    User.findByIdAndUpdate(updatedOrder.seller, { $inc: { totalSales: -
(updatedOrder.quantity || 1) } }),
    User.findByIdAndUpdate(updatedOrder.buyer, { $inc: { totalOrders: -1 } })
  ]).catch(err => console.error('[orders] counter update error:', err));
}
```

Current code — `controllers/checkout/payment.js:25-33` (same intent, *but does not restore the `$inc` quantity the same way and `dispute.js:184` omits the `$inc` and counters entirely*):

```
await Product.findByIdAndUpdate(order.product, {
  $inc: { quantity: order.quantity || 1 },
  $set: { status: PRODUCT_STATUS.ACTIVE, buyer: null, soldAt: null }
});
```

```
await Promise.all([
  User.findByIdAndUpdate(order.seller, { $inc: { totalSales: -(order.quantity ||
1) } }),
  User.findByIdAndUpdate(order.buyer, { $inc: { totalOrders: -1 } })
]).catch(() => {});
```

- Issue: The compensating "release product + roll back counters" logic is reimplemented in three places with subtle divergence — `dispute.js:184-185` only resets status/buyer/soldAt and never restores `quantity` or counters.
- Impact: A buyer-favor dispute refund leaves stock at the decremented value while order #2's cancel restores it — divergent inventory and seller stats depending on which path cancelled the order. A bug fixed in one path silently persists in the other two.

Suggested fix — one shared helper used by all three call sites:

```
// services/inventoryService.js
async function releaseProductForOrder(order, { session } = {}) {
  await Product.findByIdAndUpdate(order.product, {
    $inc: { quantity: order.quantity || 1 },
    $set: { status: PRODUCT_STATUS.ACTIVE, buyer: null, soldAt: null }
  }, { session });
  await Promise.allSettled([
    User.findByIdAndUpdate(order.seller, { $inc: { totalSales: -(order.quantity ||
1) } }, { session }),
    User.findByIdAndUpdate(order.buyer,  { $inc: { totalOrders: -1 } },
{ session }),
  ]);
}
module.exports = { releaseProductForOrder };
```

`orders/index.js:428`, `checkout/payment.js:25` and `dispute.js:184` all call `releaseProductForOrder(order)`, guaranteeing identical inventory/counter behavior on every cancel path.

ARCH-2: Token/user resolution duplicated across two middlewares

- Severity: Low
- Evidence: `middleware/auth.js:13-23` and `middleware/locals.js:33-44`

Current code — `middleware/auth.js:13-19`:

```
try {
  const { sub } = jwt.verify(token, process.env.JWT_SECRET);
  req.user = await User.findById(sub).select('-__v').lean();
  if (!req.user) {
```

```
    return res.status(401).json({ success: false, message: 'User no longer exists'
});
  }
  next();
```

Current code — `middleware/locals.js:38-44` (the same decode + `User.findById(...).select('-__v').lean()`, but it *clears the cookie* on failure instead of returning 401):

```
try {
  const { sub } = jwt.verify(token, process.env.JWT_SECRET);
  res.locals.user = await User.findById(sub).select('-__v').lean();
  req.user = res.locals.user;
} catch {
  res.clearCookie('token');
}
```

- Issue: Two copies of "verify JWT, load user, `.select('-__v').lean()`" with divergent failure behavior and divergent field hiding (both keep `googleId`, see API-3).
- Impact: Two code paths to keep in sync; a page route that also calls an API does the same `User.findById` twice (double DB hit per request).

Suggested fix — a shared resolver both middlewares delegate to:

```
// middleware/resolveUser.js
exports.userFromToken = async (token) => {
  if (!token) return null;
  try {
    const { sub } = jwt.verify(token, process.env.JWT_SECRET);
    return await User.findById(sub).select('-__v -googleId').lean();
  } catch { return null; }
};
```

`protect` returns 401 when it resolves `null`; `injectUser` clears the cookie when it resolves `null` — but the verify/query/projection lives in exactly one place (and fixes API-3 at the same time).

4.6 API design (backend)

API-1: `PATCH /api/products/:id` is overloaded with implicit side effects

- Severity: Medium
- Evidence: `controllers/product/index.js:101-170`

Current code — `controllers/product/index.js:114-139`:

```
if (k === 'quantity') {
  const quantity = parseProductQuantity(req.body[k]);
  if (!Number.isFinite(quantity)) throw new Error('Quantity must be a non-negative
number');
  product.quantity = quantity;
  if (quantity === 0) { product.status = 'sold'; product.soldAt = product.soldAt
|| new Date(); }
  if (quantity > 0 && product.status === 'sold' && req.body.status === undefined)
{
    product.status = 'active'; product.soldAt = null; product.buyer = null;
  }
  return;
}
product[k] = req.body[k];
// ...
if (req.body.status === STATUS.SOLD && oldStatus !== STATUS.SOLD) {
  product.quantity = 0;
  product.soldAt = new Date();
  if (req.body.buyerId) product.buyer = req.body.buyerId;
  User.findByIdAndUpdate(product.seller, { $inc: { totalSales: 1 } }).catch(() =>
{});
}
```

- Issue: One generic PATCH silently performs three distinct domain transitions — relist (`quantity>0` re-opens a sold listing), mark-sold (`quantity===0` or `status==='sold'`), and increments `User.totalSales` — with no explicit intent and no transactional guarantee.
- Impact: Editing only `quantity` can silently re-open a sold product; a `status:'sold'` edit double-counts `totalSales`; clients cannot tell which transition occurred, and authorization cannot distinguish "edit description" from "sell".

Suggested fix — separate descriptive edits from explicit transitions:

```
// routes/products.js
router.patch('/:id',           protect, productCtrl.updateDescriptiveFields); //
title/price/images only
router.post('/:id/mark-sold',  protect, productCtrl.markSold);               //
sets quantity=0, totalSales++
router.post('/:id/relist',     protect, productCtrl.relist);                 //
ACTIVE, clears buyer/soldAt

// controllers/product/index.js
const DESCRIPTIVE =
['title','description','price','category','condition','images','location'];
exports.updateDescriptiveFields = async (req, res) => {
  // ...authorize, then assign ONLY DESCRIPTIVE keys; never touch
```

```
status/quantity/totalSales
};
```

Each transition is explicit, authorizable and idempotent; `totalSales` is bumped in exactly one place.

API-2: Inconsistent HTTP status codes; client errors returned as 500

- Severity: Medium
- Evidence: `controllers/orders/index.js:308`, `controllers/product/index.js:184`, `controllers/ai/index.js:111`

Current code — `controllers/orders/index.js:307-309` (and the same pattern in ~20 catch blocks):

```
} catch (err) {
  res.status(500).json({ success: false, message: err.message });
}
```

Current code — `controllers/ai/index.js:109-112` (a missing `GROQ_API_KEY` *server config* and a bad image URL *client error* both collapse to 500):

```
} catch (err) {
  console.error('AI error:', err.message);
  res.status(500).json({ success: false, message: err.message || 'AI service
error' });
}
```

- Issue: No distinction between user error (4xx) and system error (5xx); a not-found, an unauthorized, a validation failure and an internal crash all return `500 { message: err.message }`.
- Impact: Clients cannot programmatically branch on error type; internal text (Mongoose cast/validation messages) leaks (also VAL-2). An invalid `entityId` or unconfigured API key is reported as a server fault.

Suggested fix — typed errors + one error middleware:

```
// utils/errors.js
class AppError extends Error { constructor(msg, status){ super(msg); this.status =
status; } }
const badRequest = (m) => new AppError(m, 400);
const notFound   = (m='Not found') => new AppError(m, 404);

// app.js (last middleware)
app.use((err, req, res, next) => {
  const status = err.status || 500;
  if (status >= 500) console.error(err);
  res.status(status).json({ success: false, message: status >= 500 ? 'Internal
```

```
server error' : err.message });
});
```

Controllers `throw notFound('Order not found')` / `next(err)`; only genuine 5xx reach the client as a generic message.

API-3: `googleId` leaked because `.lean()` bypasses the schema `toJSON` transform

- Severity: Low
- Evidence: `controllers/auth/index.js:35-37`, `middleware/auth.js:15`, `models/User.js:53-60`

Current code — `controllers/auth/index.js:35-37`:

```
const getMe = (req, res) => {
  res.json({ success: true, data: req.user });
};
```

Current code — `middleware/auth.js:15` (loads the user with `.lean()`, which returns a plain object so the schema transform never runs):

```
req.user = await User.findById(sub).select('-__v').lean();
```

Current code — `models/User.js:53-60` (the transform that is meant to hide `googleId` but is skipped by `.lean()`):

```
UserSchema.set('toJSON', {
  virtuals: true,
  transform: (doc, ret) => {
    delete ret.__v;
    delete ret.googleId;
    return ret;
  },
});
```

- Issue: `.lean()` returns a POJO, so `toJSON` (the only place `googleId` is stripped) never executes; `select('-__v')` removes `__v` but keeps `googleId`. `GET /api/auth/me` therefore returns the raw Google subject id.
- Impact: An internal identity provider id is exposed to the browser/any client, defeating the schema's intended hiding.

Suggested fix — strip the field in the query projection (works with `.lean()`):

```
// middleware/auth.js  (and middleware/locals.js)
req.user = await User.findById(sub).select('-__v -googleId').lean();
```

Projecting the field out at the database layer guarantees it is gone regardless of `.lean()`; ideally combined with the shared resolver from ARCH-2.

4.7 Database & N+1 queries (backend)

DB-1: N+1 query in chat inbox

- Severity: High
- Evidence: `controllers/chat/index.js:43-60`

Current code — `controllers/chat/index.js:43-46`:

```
const results = [];
for (let c of convs) {
  const unread = await Message.countDocuments({ conversationId: c._id, sender: {
$ne: userId }, isRead: false }).catch(() => 0);
  c.unreadCount = unread;
```

- Issue: One additional `countDocuments` is fired per conversation in a sequential `await` loop.
- Impact: A user with N conversations causes N+1 round-trips and serially blocks; inbox latency grows linearly (e.g. 50 conversations ≈ 51 queries instead of 2).

Suggested fix — single grouped aggregation, then map back:

```
const ids = convs.map(c => c._id);
const counts = await Message.aggregate([
  { $match: { conversationId: { $in: ids }, isRead: false, sender: { $ne: userId }
} },
  { $group: { _id: '$conversationId', count: { $sum: 1 } } },
]);
const byConv = Object.fromEntries(counts.map(c => [String(c._id), c.count]));
for (const c of convs) c.unreadCount = byConv[String(c._id)] || 0;
```

Two queries total regardless of conversation count.

DB-2: Full-collection scan + N+1 in rating recalculation

- Severity: High
- Evidence: `controllers/rating/index.js:66-78`, `controllers/rating/index.js:155-163`

Current code — `controllers/rating/index.js:66-72` (reloads *all* ratings for the entity on every single submission just to average them):

```
const allRatings = await Rating.find({
  ratedEntity: entityType,
  entityId,
});
const totalScore = allRatings.reduce((sum, r) => sum + r.score, 0);
const average = (totalScore / allRatings.length).toFixed(2);
```

Current code — `controllers/rating/index.js:157-163` (`syncAllRatings` loads every user, then `updateSellerRating` runs another `Product.find` per user):

```
const users = await User.find({});
let count = 0;
for (const user of users) {
  await updateSellerRating(user._id);
  count++;
}
```

- Issue: `submitRating` does an unbounded `Rating.find` and in-memory reduce per write; `syncAllRatings` is O(users) loaded into memory with O(products-per-user) queries each.
- Impact: Rating writes slow down as review count grows; the admin sync endpoint is O(users × products) and will time out / spike memory on any non-trivial dataset.

Suggested fix — aggregate in the database:

```
// submitRating: compute avg/count in one query
const [agg] = await Rating.aggregate([
  { $match: { ratedEntity: entityType, entityId: new
mongoose.Types.ObjectId(entityId) } },
  { $group: { _id: null, avg: { $avg: '$score' }, count: { $sum: 1 } } },
]);
const average = (agg?.avg || 0).toFixed(2);
const ratingCount = agg?.count || 0;

// syncAllRatings: stream users with a cursor instead of loading all
for await (const user of User.find({}, { _id: 1 }).cursor()) {
  await updateSellerRating(user._id);
}
```

The average is computed by Mongo (no document transfer); the cursor bounds memory on the admin sync.

DB-3: No transactions across multi-collection money/order flows

- Severity: High
- Evidence: `controllers/orders/index.js:106-229`, `controllers/orders/index.js:481-518`, `controllers/checkout/payment.js:78-120`

Current code — `controllers/checkout/payment.js:78-111` (payment → order → wallet → transaction, five separate awaits, no session):

```
payment.status = 'PAID';
payment.paidAt = new Date();
await payment.save();

const order = await Order.findById(payment.order);
if (order) { order.status = ORDER_STATUS.PENDING; /* push timeline */ await
order.save(); }

let wallet = await Wallet.findOne({ user: payment.seller });
if (!wallet) { wallet = await Wallet.create({ user: payment.seller }); }
wallet.pendingBalance += payment.amount;
await wallet.save();

await WalletTransaction.create({ wallet: wallet._id, user: payment.seller, type:
'DEPOSIT',
  amount: payment.amount, status: 'COMPLETED', /* ... */ });
```

- Issue: MongoDB multi-document writes here are not atomic and there is no `session`/`withTransaction`. A crash after `payment.save()` but before `wallet.save()` leaves a PAID payment with no wallet credit; the order-creation flow's "rollback" is a best-effort `catch` (`controllers/orders/index.js:249-267`).
- Impact: Financial/inventory corruption under partial failure — the most serious bug class for a marketplace (paid-but-not-credited, or counters bumped on a rolled-back order).

Suggested fix — wrap the flow in a session and do side effects post-commit:

```
const session = await mongoose.startSession();
await session.withTransaction(async () => {
  await Payment.updateOne({ _id: payment._id, status: 'PENDING' },
    { $set: { status: 'PAID', paidAt: new Date() } }, { session });
  await Order.updateOne({ _id: payment.order }, { $set: { status:
ORDER_STATUS.PENDING } }, { session });
  await Wallet.updateOne({ user: payment.seller },
    { $inc: { pendingBalance: payment.amount } }, { upsert: true, session });
  await WalletTransaction.create([{ /* ... */ }], { session });
});
session.endSession();
await sendNotification(/* ... */); // post-commit, non-transactional
```

All four writes commit or roll back together (requires a replica set / Atlas); notifications/socket emits run only after commit.

DB-4: Wallet credit is a non-atomic read-modify-write; webhook + poll can double-credit

- Severity: Medium
- Evidence: `controllers/checkout/payment.js:99-100` (webhook) and `controllers/checkout/payment.js:233-234` (poll)

Current code — `controllers/checkout/payment.js:99-100`:

```
wallet.pendingBalance += payment.amount;
await wallet.save();
```

Current code — `controllers/checkout/payment.js:69-71` (the only concurrency guard is a non-atomic read of `payment.status`):

```
if (payment.status !== 'PENDING') {
  return res.json({ success: true, message: 'Already processed' });
}
```

- Issue: `payment.status !== 'PENDING'` check and the later `payment.save()` are not atomic, and `wallet.pendingBalance += amount; await wallet.save()` is a read-modify-write. The webhook (`:55`) and the client poll path (`checkPaymentViaSePay:205-234`) can both pass the guard for the same payment concurrently.
- Impact: The seller wallet can be credited twice for one payment if the webhook and a client poll race (lost-update on `pendingBalance`).

Suggested fix — atomic guarded status flip + `$inc` wallet:

```
const claimed = await Payment.findOneAndUpdate(
  { _id: payment._id, status: 'PENDING' },
  { $set: { status: 'PAID', paidAt: new Date() } },
  { new: true }
);
if (!claimed) return res.json({ success: true, message: 'Already processed' });

await Wallet.updateOne({ user: claimed.seller },
  { $inc: { pendingBalance: claimed.amount } }, { upsert: true });
```

Only the first request whose `status` is still `PENDING` wins the `findOneAndUpdate`; the loser short-circuits, so the wallet is credited exactly once.

DB-IDX-1: Improper / undisciplined index declarations (duplicate unique indexes, low-cardinality singles, prod `autoIndex` on)

- Severity: Medium

- Evidence: `models/Payment.js:14`, `models/Payment.js:19`, `models/Payment.js:43-57`, `models/Product.js:38`, `models/Product.js:45`, `models/User.js:7`, `models/User.js:32`, `config/database.js:3-51`, `config/database.js:59-64`

Yes — there is improper indexing. The schemas mix inline `unique: true, index: true` with redundant `Schema.index(...)` calls on the same field, scatter single-field indexes on low-cardinality booleans, and leave `autoIndex` on in production. The strongest proof is that `config/database.js` ships a runtime "repair" routine that drops and recreates the `sepayPaymentId` / `bankTransactionId` indexes on every boot precisely because the duplicate declarations below collided in production.

Current code — `models/Payment.js:14-19` (each field carries BOTH `unique: true` and `index: true` — `unique` already builds an index, so this is redundant):

```
  paymentCode: { type: String, sparse: true, unique: true, index: true },
  ...
  sepayPaymentId: { type: String, sparse: true, unique: true, index: true },
```

Current code — `models/Payment.js:43-57` (a SECOND, different unique index is then declared on `sepayPaymentId`, and a unique index on `bankTransactionId` which has no inline declaration — the two `sepayPaymentId` indexes conflict, forcing the boot-time `repairPaymentIndexes` hack in `config/database.js`):

```
PaymentSchema.index(
  { sepayPaymentId: 1 },
  {
    unique: true,
    partialFilterExpression: { sepayPaymentId: { $type: 'string' } }
  }
);

PaymentSchema.index(
  { bankTransactionId: 1 },
  {
    unique: true,
    partialFilterExpression: { bankTransactionId: { $type: 'string' } }
  }
);
```

Current code — `models/Product.js:34-46` (`status` already covered by the compound `{ status: 1, createdAt: -1 }` / `{ category: 1, status: 1 }` at lines 78-79, so the inline single is redundant; `reported` is a low-cardinality boolean indexed alone but never used as a sole query path — admin moderation queries the `Report` collection):

```
    status: {
      type: String,
      enum: Object.values(PRODUCT_STATUS),
      default: PRODUCT_STATUS.ACTIVE,
      index: true,
    },

    reported: {
      type: Boolean,
      default: false,
      index: true,
    },
```

Current code — `models/User.js:7,32` (`googleId` is `unique: true, index: true` — redundant; `banned` is a low-cardinality boolean indexed alone):

```
    googleId: { type: String, required: true, unique: true, index: true },
    ...
    banned: { type: Boolean, default: false, index: true },
```

Current code — `config/database.js:59-64` (connection options never set `autoIndex: false`, so Mongoose auto-builds every declared index on each process start, including in production):

```
const options = {
  retryWrites: true,
  maxPoolSize: 10,
  serverSelectionTimeoutMS: 20000,
  socketTimeoutMS: 45000,
};
```

- Issue: Indexes are declared "by feel" rather than by query: (1) `unique: true` is duplicated with `index: true` on the same field (`Payment.paymentCode`, `Payment.sepayPaymentId`, `User.googleId`) — `unique` already implies an index; (2) `Payment.sepayPaymentId` has two competing definitions (inline non-partial unique vs. a partial-filter unique), which is exactly why `repairPaymentIndexes` has to drop/recreate them at startup; (3) scattered single-field indexes on low-cardinality booleans (`Product.reported`, `User.banned`) and on `Product.status` (already a prefix of two compound indexes) are unused or redundant; (4) `autoIndex` is left on in production; (5) no naming convention — a mix of auto-named inline indexes and ad-hoc `Schema.index` calls with no explicit `name`.
- Impact: Write amplification (every Payment/Product/User insert/update maintains several indexes that no query selects on), wasted RAM holding redundant/low-selectivity indexes resident, a fragile boot-time index-repair hack masking the duplicate-definition bug, and migration/observability pain

from inconsistent index names. Low-cardinality boolean indexes (`reported`, `banned`) are barely more selective than a collection scan, so they cost writes without speeding reads.

Suggested fix — index only fields used in real `find`/`sort`/filter paths, prefer compound over many singles, drop low-cardinality singletons, declare each index once with an explicit name, and disable `autoIndex` in production:

```
// models/Payment.js — declare each unique constraint ONCE (partial), drop inline
index:true
  paymentCode:    { type: String },
  sepayPaymentId: { type: String },
  bankTransactionId: { type: String },
// keep ONLY these (remove inline unique/index on the three fields above):
PaymentSchema.index({ paymentCode: 1 }, { unique: true, name: 'uniq_paymentCode',
  partialFilterExpression: { paymentCode: { $type: 'string' } } });
PaymentSchema.index({ sepayPaymentId: 1 }, { unique: true, name:
'uniq_sepayPaymentId',
  partialFilterExpression: { sepayPaymentId: { $type: 'string' } } });
PaymentSchema.index({ bankTransactionId: 1 }, { unique: true, name:
'uniq_bankTransactionId',
  partialFilterExpression: { bankTransactionId: { $type: 'string' } } });
// then repairPaymentIndexes() in config/database.js can be deleted.

// models/Product.js — drop the redundant/low-cardinality singles:
//   status: { ... }            // remove `index: true` — covered by the compound
below
//   reported: { ... }          // remove `index: true` — low-cardinality boolean
ProductSchema.index({ status: 1, createdAt: -1 }, { name: 'status_createdAt' });

// models/User.js — `unique` already creates the index; drop the low-cardinality
boolean:
  googleId: { type: String, required: true, unique: true }, // no separate
index:true
  banned:   { type: Boolean, default: false },              // no index:true

// config/database.js — disable autoIndex in production:
  const options = {
    retryWrites: true,
    maxPoolSize: 10,
    serverSelectionTimeoutMS: 20000,
    socketTimeoutMS: 45000,
    autoIndex: process.env.NODE_ENV !== 'production',
  };
```

Rule: index only columns that appear in a real query/sort/filter, prefer a single compound index over several single-field indexes for multi-field query+sort paths, never index a low-cardinality field (boolean/status/role)

on its own, declare each index exactly once with a consistent explicit `name`, and set `autoIndex: false` in production (build indexes via a controlled migration instead).

4.8 Performance (backend)

PERF-1: Unbounded queries on admin/sync and inbox endpoints

- Severity: Medium
- Evidence: `controllers/rating/index.js:157`, `controllers/chat/index.js:37-41`

Current code — `controllers/chat/index.js:37-41` (loads *all* conversations for a user, no pagination, then the DB-1 loop):

```
const convs = await Conversation.find({ participants: userId })
  .populate('participants', 'name nickname avatar')
  .populate('product', 'title images price seller')
  .sort('-updatedAt')
  .lean();
```

- Issue: No upper bound on result sets for endpoints that grow with usage (`User.find({})` at `rating/index.js:157`, full conversation list here).
- Impact: Memory spikes and slow responses at scale; the admin sync can time out (compounded by DB-2).

Suggested fix — paginate the inbox (and cursor-batch the sync per DB-2):

```
const page  = Math.max(1, Number(req.query.page)  || 1);
const limit = Math.min(50, Math.max(1, Number(req.query.limit) || 20));
const convs = await Conversation.find({ participants: userId })
  .sort('-updatedAt').skip((page - 1) * limit).limit(limit)
  .populate('participants', 'name nickname avatar')
  .populate('product', 'title images price seller').lean();
```

Bounds the working set and the per-conversation unread aggregation regardless of how many threads a user has.

PERF-2: External AI / SePay calls are uncached and synchronous in the request path

- Severity: Medium
- Evidence: `controllers/ai/index.js:96-105`, `controllers/checkout/payment.js:172`

Current code — `controllers/checkout/payment.js:170-180` (every client poll tick hits the SePay API for the last 100 transactions):

```
let transactions = [];
try {
  transactions = await sepayService.getRecentTransactions(1, 100);
} catch (err) {
  console.error('[Payment Check] SePay API error:', err.message);
  return res.json({ success: true, status: 'PENDING', message: 'Checking payment
status...' });
}
```

Current code — `controllers/ai/index.js:96-105` (no cache; a vision failure issues a *second* full Groq call):

```
description = AI_PROVIDER === 'gemini'
  ? await generateWithGemini({ prompt, imageUrl, maxTokens })
  : await generateWithGroq({ prompt, imageUrl, maxTokens });
} catch (err) {
  if (!imageUrl || AI_PROVIDER !== 'groq') throw err;
  description = await generateWithGroq({ prompt: textOnlyPrompt, imageUrl: '',
maxTokens });
}
```

- Issue: Payment-status polling re-fetches 100 transactions from SePay on every client tick with no throttle/cache; AI description regenerates from scratch every request.
- Impact: Third-party rate-limit/billing exposure and event-loop pressure under load; a page with 5 buyers polling = 5× SePay calls/second.

Suggested fix — short-TTL cache the SePay transaction list:

```
// services/sepayService.js
let _cache = { at: 0, data: [] };
async function getRecentTransactionsCached() {
  if (Date.now() - _cache.at < 5000) return _cache.data;       // 5s TTL
  _cache = { at: Date.now(), data: await getRecentTransactions(1, 100) };
  return _cache.data;
}
```

Many concurrent pollers now share one upstream call per 5s window; add a similar TTL/dedup on `/api/ai/describe`.

PERF-4: Silent error-swallowing in financial flows; `console.*` is the only logging

- Severity: Low
- Evidence: `controllers/orders/index.js:170-173`, `controllers/checkout/payment.js:33`

Current code — `controllers/orders/index.js:170-173` (counter failures are logged-and-forgotten on a money flow):

```
Promise.all([
  User.findByIdAndUpdate(sellerId, { $inc: { totalSales: orderQuantity } }),
  User.findByIdAndUpdate(buyerId, { $inc: { totalOrders: 1 } })
]).catch(err => console.error('[orders] counter update error:', err));
```

Current code — `controllers/checkout/payment.js:30-33` (counter rollback errors swallowed entirely):

```
await Promise.all([
  User.findByIdAndUpdate(order.seller, { $inc: { totalSales: -(order.quantity ||
1) } }),
  User.findByIdAndUpdate(order.buyer, { $inc: { totalOrders: -1 } })
]).catch(() => {});
```

- Issue: No structured/leveled logger; failures in counter/rollback paths are dropped to `console` or `.catch(() => {})` with no alerting.
- Impact: Silent data drift on `totalSales`/`totalOrders` that nobody notices in production; no way to grep/aggregate errors.

Suggested fix — a leveled logger and never empty-catch a money path:

```
const logger = require('pino')();
Promise.allSettled([ /* ... */ ]).then(rs =>
  rs.filter(r => r.status === 'rejected')
    .forEach(r => logger.error({ err: r.reason, orderId: order._id }, 'counter
update failed')));
```

`pino`/`winston` gives queryable structured logs; `.catch(() => {})` is replaced by an explicit, logged failure.

4.9 Constants, enums, config (backend)

CONST-1: Payment-status magic strings bypass the centralized enums; no `PAYMENT_STATUS`

- Severity: Low
- Evidence: `controllers/checkout/payment.js:60`, `:69`, `:78`; `controllers/orders/index.js:496`; `controllers/product/index.js:11`

Current code — `controllers/checkout/payment.js:60-78` (no central enum for payment status — `'PAID'`, `'SUCCESS'`, `'PENDING'`, `'EXPIRED'` are all string literals):

```
if (status !== 'PAID' && status !== 'SUCCESS') {
  return res.status(400).json({ success: false, message: 'Ignored status' });
}
const payment = await Payment.findOne({ paymentCode });
if (!payment) { return res.status(404).json({ success: false, message: 'Payment
not found' }); }
if (payment.status !== 'PENDING') { return res.json({ success: true, message:
'Already processed' }); }
// ...
payment.status = 'PAID';
```

Current code — `controllers/product/index.js:11` (`'active'` literal instead of `PRODUCT_STATUS.ACTIVE`, while the same file's sibling `orders` controller imports the enum):

```
const filter = { status: 'active', $or: [{ quantity: { $gt: 0 } }, { quantity: {
$exists: false } }] };
```

- Issue: `config/appConstants.js` freezes order/product/role enums but has **no `PAYMENT_STATUS`**, so the most financially sensitive state machine is stringly-typed; `'active'` and `req.user.role !== 'admin'` (`product/index.js:106`) also bypass the existing enums.
- Impact: Typo-prone and refactor-unsafe on the payment path; a misspelled `'PAdID'` silently never matches and a payment hangs forever.

Suggested fix — add the enum and consume it:

```
// config/appConstants.js
const PAYMENT_STATUS = Object.freeze({
  PENDING: 'PENDING', PAID: 'PAID', EXPIRED: 'EXPIRED', FAILED: 'FAILED',
});
module.exports = { /* ...existing..., */ PAYMENT_STATUS };

// controllers/checkout/payment.js
const { PAYMENT_STATUS } = require('../../config/appConstants');
if (payment.status !== PAYMENT_STATUS.PENDING) return res.json({ success: true,
message: 'Already processed' });
payment.status = PAYMENT_STATUS.PAID;
```

Single source of truth for payment state; also replace `status: 'active'` with `PRODUCT_STATUS.ACTIVE` and `role !== 'admin'` with `USER_ROLES.ADMIN`.

4.10 Validation & error handling (backend)

VAL-1: No schema validation layer; ad-hoc manual checks, incomplete coverage

- Severity: Medium

- Evidence: `controllers/product/index.js:77-90`, `controllers/rating/index.js:14-29`, `controllers/orders/index.js:31-45`

Current code — `controllers/product/index.js:77-90` (`price`, `category`, `condition` are never validated before `Product.create`; only `quantity` is checked):

```
const createProduct = async (req, res) => {
  try {
    const { title, description, price, category, condition, images, location } =
req.body;
    const quantity = parseProductQuantity(req.body.quantity ?? 1);
    if (!Number.isFinite(quantity) || quantity < 1) {
      return res.status(400).json({ success: false, message: 'Quantity must be at
least 1' });
    }
    const product = await Product.create({
      title, description, price, quantity, category, condition,
      images: images || [], location: location || {}, seller: req.user._id,
    });
```

- Issue: Validation is scattered hand-written `if`s, inconsistent between controllers and incomplete — `price` type/range, `category` membership and `title` length are delegated to Mongoose throwing (then surfaced as a raw 400, see VAL-2). No `joi`/`zod`/`express-validator` in `package.json`.
- Impact: Inconsistent 400 shapes, easy-to-miss fields, no reusable/testable validation; a negative or non-numeric `price` reaches the DB layer.

Suggested fix — a schema + a validation middleware:

```
// validation/productSchemas.js
const { z } = require('zod');
const createProductSchema = z.object({
  title: z.string().min(3).max(120),
  price: z.number().positive(),
  category: z.enum(PRODUCT_CATEGORIES),
  condition: z.enum(PRODUCT_CONDITIONS),
  quantity: z.coerce.number().int().min(1).default(1),
  images: z.array(z.string().url()).optional(),
});

// middleware/validate.js
const validate = (schema) => (req, res, next) => {
  const r = schema.safeParse(req.body);
  if (!r.success) return res.status(400).json({ success: false, errors:
r.error.flatten() });
  req.body = r.data; next();
};
```

```
// routes/products.js
router.post('/', protect, validate(createProductSchema),
productCtrl.createProduct);
```

One declarative schema enforces every field with a uniform 400 shape, reused on create and update.

VAL-2: Raw internal error messages returned to the client

- Severity: Medium
- Evidence: `controllers/chat/index.js:64`, `controllers/orders/index.js:308`, `controllers/checkout/index.js` (and ~20 more catch blocks)

Current code — `controllers/chat/index.js:62-65`:

```
res.json({ success: true, data: results });
} catch (error) {
  res.status(500).json({ success: false, message: error.message });
}
```

- Issue: `error.message` (Mongoose cast/validation text, library internals) is forwarded verbatim in nearly every catch.
- Impact: Information disclosure (schema field names, internal state) and inconsistent client-facing messages.

Suggested fix — route everything through the central handler from API-2:

```
} catch (error) { next(error); }   // handler logs detail server-side, returns
generic 500 message
```

The error middleware logs `error` server-side and responds `{ success:false, message:'Internal server error' }` for 5xx, exposing only intentional 4xx messages.

4.11 Security (backend)

SEC-1: Broken conversation authorization (`ObjectId.includes` always false) → IDOR

- Severity: Critical
- Evidence: `controllers/chat/index.js:77` and `controllers/chat/index.js:110`

Current code — `controllers/chat/index.js:74-79`:

```
const conv = await Conversation.findById(id);
if (!conv) return res.status(404).json({ success: false, message: 'Conversation
not found' });
```

```
if (!conv.participants.includes(userId)) {
  return res.status(403).json({ success: false, message: 'Forbidden' });
}
```

Current code — `controllers/chat/index.js:108-110` (`sendMessage`, same broken guard):

```
const conv = await Conversation.findById(id);
if (!conv) return res.status(404).json({ success: false, message: 'Conversation
not found' });
if (!conv.participants.includes(userId)) return res.status(403).json({ success:
false, message: 'Forbidden' });
```

- Issue: `conv.participants` is an array of Mongoose `ObjectId` instances and `userId = req.user._id` is also an `ObjectId`. `Array.prototype.includes` uses strict reference equality (SameValueZero), which is never true for two distinct ObjectId objects — so `!conv.participants.includes(userId)` is always `true`... and the function returns `403` for the legitimate owner too, *unless* `participants` happens to be unpopulated string-equal — i.e. the participant check is non-functional, and worse, the only thing stopping access is whether the comparison accidentally matches. There is no reliable participant check on `getMessages`/`sendMessage`.
- Impact: A logged-in user can read any conversation's messages and post into any conversation by enumerating/guessing a `conversationId` (IDOR). Private buyer↔seller chats and the auto-generated order messages (which contain shipping addresses, see `controllers/orders/index.js:186-193`) are exposed.

Suggested fix — scope the query so a non-participant simply gets 404:

```
const conv = await Conversation.findOne({ _id: id, participants: userId });
if (!conv) return res.status(404).json({ success: false, message: 'Conversation
not found' });
// no separate participants.includes check needed
```

Mongoose `participants: userId` does correct ObjectId equality in the query; a non-participant never matches, so messages cannot be read or sent cross-user. Apply to both `getMessages` and `sendMessage`.

SEC-2: No CSRF protection on cookie-authenticated state-changing routes

- Severity: High
- Evidence: `controllers/auth/index.js:10-17`, all `POST/PATCH` API routes

Current code — `controllers/auth/index.js:10-17` (auth is a cookie with `sameSite:'lax'`, no CSRF token anywhere):

```
const sendTokenCookie = (res, token) => {
  res.cookie('token', token, {
    httpOnly: true,
    secure:   process.env.NODE_ENV === 'production',
    sameSite: 'lax',
    maxAge:   7 * 24 * 60 * 60 * 1000,
  });
};
```

- Issue: Every state-changing endpoint (`POST /api/orders`, `PATCH /api/products/:id`, `POST /api/ratings`, `POST /api/chat/:id/messages`) authenticates solely via this cookie. `sameSite=lax` mitigates but does not fully prevent CSRF (top-level GET-initiated navigations, future relaxation), and there is no Origin/Referer check (`grep` shows no `csurf`/double-submit).
- Impact: A cross-site page could trigger orders/messages on behalf of a logged-in user.

Suggested fix — double-submit CSRF token (or require a Bearer header + Origin check for the API):

```
// issue on page render
res.cookie('csrf', token, { sameSite: 'lax' }); // readable by JS
// middleware on state-changing routes
const csrfGuard = (req, res, next) => {
  if (req.get('x-csrf-token') && req.get('x-csrf-token') === req.cookies.csrf)
return next();
  return res.status(403).json({ success: false, message: 'CSRF check failed' });
};
app.use(['/api/orders','/api/products','/api/ratings','/api/chat'], csrfGuard);
```

The attacker site cannot read the `csrf` cookie value to echo it in the header, so forged cross-site writes fail.

SEC-3: Socket.IO accepts any origin while authenticating via cookies

- Severity: Medium
- Evidence: `utils/socketServer.js:7` and `utils/socketServer.js:28-31`

Current code — `utils/socketServer.js:7`:

```
io = new Server(server, { cors: { origin: '*' } });
```

Current code — `utils/socketServer.js:28-31` (a failed token verify still calls `next()`, so the socket connects anonymously instead of being rejected):

```
} catch (err) {
  // Still allow connection but socket.userId will be missing
  next();
}
```

- Issue: Wildcard CORS on a socket server that derives identity from the `token` cookie; an invalid/missing token does not reject the connection.
- Impact: Any website can open an authenticated socket in the visitor's browser context (combined with the weak CSRF posture, SEC-2); unauthenticated sockets are allowed to connect and join rooms.

Suggested fix — restrict origin and reject unauthenticated sockets:

```
io = new Server(server, { cors: { origin: process.env.CLIENT_URL, credentials:
true } });
io.use((socket, next) => {
  try {
    const { token } = parseCookies(socket.handshake.headers.cookie || '');
    socket.userId = jwt.verify(token, process.env.JWT_SECRET).sub;
    return next();
  } catch {
    return next(new Error('unauthorized'));   // reject, do not connect
  }
});
```

Only the known client origin can connect and only with a valid token.

SEC-4: `getPaymentStatus` does not verify the requester owns the payment

- Severity: Medium
- Evidence: `controllers/checkout/payment.js:36-49`

Current code — `controllers/checkout/payment.js:36-49`:

```
exports.getPaymentStatus = async (req, res) => {
  try {
    const payment = await Payment.findById(req.params.id);
    if (!payment) {
      return res.status(404).json({ success: false, message: 'Payment not
found' });
    }
    if (payment.status === 'PENDING' && new Date() > payment.expiredAt) {
      payment.status = 'EXPIRED';
      await payment.save();
      await cancelOrderAndRestoreStock(payment.order);
    }
    res.json({ success: true, data: { status: payment.status } });
```

- Issue: `GET /api/payments/:id/status` returns any payment's status by id with no `payment.buyer === req.user._id` check (unlike `checkPaymentViaSePay`). Worse, it triggers

`cancelOrderAndRestoreStock` as a side effect — so an unauthenticated/other user polling an id can *cancel someone else's order* once it expires.

- Impact: Payment-status enumeration plus a write side effect (order cancellation) reachable by a non-owner.

Suggested fix — scope to the owner and drop the destructive side effect from a GET:

```
const payment = await Payment.findOne({ _id: req.params.id, buyer:
req.user._id });
if (!payment) return res.status(404).json({ success: false, message: 'Payment not
found' });
// move expiry/cancel into a dedicated job or the SePay-verify path, not a GET
status read
res.json({ success: true, data: { status: payment.status } });
```

Only the buyer can read their payment, and a read no longer mutates order state.